EvoArena — Project Progress Report 3

Rohan Upadhyay | CSCI 411: Senior Seminar I | April 8, 2026

GitHub: https://github.com/RyanX5/evoarena

---

1. Project Overview

EvoArena is an agent-based evolutionary simulation where autonomous agents compete in a 2D arena and develop combat and survival behaviors through evolution. Each agent is controlled by a small neural network whose weights are shaped over generations using a genetic algorithm. Successful agents reproduce, passing their strategies to the next generation.

The intended use case is demonstrating how complex behaviors can emerge from simple agents under selection pressure, without any hand-coded rules. The project covers core CS concepts including neural computation, evolutionary algorithms, agent-based modeling, and system design.

Current stage: Final pre-presentation milestone. All core simulation systems have been operational since the midterm, with fitness tracking, crossover reproduction, obstacle collision, and alpha agent visualization added in Report 2. This report documents three new additions: headless simulation mode, champion agent save and replay, and a parameter sensitivity analysis across mutation rate, population size, and elite percentage.

---

2. Implementation Status

All modules from Reports 1 and 2 remain complete. The following table reflects the full current state of the project.

Module	Status	Description
`simulation/arena.py`	Complete	Arena environment, step loop, obstacle placement, agent management
`simulation/agent.py`	Complete	Neural-network-driven decisions, combat, fitness tracking, obstacle collision
`simulation/neural_network.py`	Complete	8→12→4 feedforward NN, forward pass, weight serialization
`visualization/visualizer.py`	Complete	Real-time Pygame rendering, health bars, HUD, alpha agent highlight, pinned champion mode
`visualization/fitness_graph.py`	Complete	Per-generation fitness tracking, saved to `fitness.png` via matplotlib
`evolution/evolve.py`	Complete	Elitist selection, Gaussian mutation, crossover reproduction

Module	Status	Description
`main.py`	Complete	Generation loop, headless mode, champion saving, replay mode
`experiments/sweep.py`	Complete	Headless parameter sensitivity sweep across 3 variables

---

3. Key Implementations Since Report 2

3.1 Headless Mode

Prior to this milestone, every simulation run required an open Pygame window. This made batch experimentation impractical, running 9 configurations of 50 generations each with a live window would be both slow and visually cluttered.

Headless mode was added via two new CLI flags in `main.py` using Python's `argparse`:

```
parser.add_argument("--headless", action="store_true")
parser.add_argument("--generations", type=int, default=None)
```

When `--headless` is set, the Pygame import is skipped entirely using a lazy import pattern. `Visualizer` is only imported inside the `if not args.headless` branch. This avoids any pygame initialization overhead and prevents display errors on machines without a screen.

```
viz = None
if not args.headless:
    from visualization.visualizer import Visualizer
    viz = Visualizer(arena=dummy_arena, fps=config.FPS)
    viz.init()
```

The generation loop checks `if viz:` before any rendering call, so the same loop runs in both modes without branching logic scattered throughout. The `--generations N` flag provides a clean stopping condition for scripted runs.

Usage:

```
python main.py                          # normal visual mode
python main.py --headless               # no window, runs fast
python main.py --headless --generations 50
```

Console output replaces the HUD in headless mode, printing best fitness, average fitness, and survivor count after each generation:

```
PS C:\Users\Rohan\Documents\evoarena> python .\main.py --headless --generations 10
EvoArena
  Population : 30
  Arena      : 800x600
  Mode       : headless

Generation: 1
  best=921.1  avg=-769.3  survivors=11
  [champion saved] champions/gen_001.npy
  [graph] saved to fitness.png  (best=921.1, avg=-769.3)
Generation: 2
  best=1225.0  avg=-237.3  survivors=12
  [champion saved] champions/gen_002.npy
  [graph] saved to fitness.png  (best=1225.0, avg=-237.3)
Generation: 3
  best=1064.7  avg=-135.4  survivors=9
  [champion saved] champions/gen_003.npy
  [graph] saved to fitness.png  (best=1064.7, avg=-135.4)
Generation: 4
  best=992.1  avg=-105.0  survivors=6
  [champion saved] champions/gen_004.npy
  [graph] saved to fitness.png  (best=992.1, avg=-105.0)
Generation: 5
  best=1287.1  avg=777.8  survivors=6
  [champion saved] champions/gen_005.npy
  [graph] saved to fitness.png  (best=1287.1, avg=777.8)
Generation: 6
  best=1310.0  avg=698.5  survivors=7
  [champion saved] champions/gen_006.npy
  [graph] saved to fitness.png  (best=1310.0, avg=698.5)
Generation: 7
  best=940.7  avg=503.6  survivors=4
  [champion saved] champions/gen_007.npy
  [graph] saved to fitness.png  (best=940.7, avg=503.6)
Generation: 8
  best=1350.0  avg=579.5  survivors=8
  [champion saved] champions/gen_008.npy
  [graph] saved to fitness.png  (best=1350.0, avg=579.5)
Generation: 9
  best=683.6  avg=8.1  survivors=6
  [champion saved] champions/gen_009.npy
  [graph] saved to fitness.png  (best=683.6, avg=8.1)
Generation: 10
  best=1321.1  avg=247.4  survivors=8
  [champion saved] champions/gen_010.npy
  [graph] saved to fitness.png  (best=1321.1, avg=247.4)
Done.
PS C:\Users\Rohan\Documents\evoarena>
```

Fig 1: Headless mode running 10 generations with per-generation fitness output

3.2 Champion Save and Replay

A key missing capability was the ability to revisit the best agent from a past generation. Without saving weights, every run was ephemeral andthere was no way to compare early-generation behavior against late-generation behavior, or to watch a trained agent perform against opponents.

Saving: At the end of each generation, the highest-fitness surviving agent is identified and its weight vector is serialized using `numpy.save`:

```
def save_champion(agent: Agent, gen: int):
    os.makedirs("champions", exist_ok=True)
    path = f"champions/gen_{gen:03d}.npy"
    np.save(path, agent.brain.get_weights())
```

This produces one `.npy` file per generation (e.g. `champions/gen_042.npy`), containing the 160-element flat weight vector of the champion's neural network.

Replaying: The `--replay` flag loads a saved champion and runs it in a visual arena against fresh randomly-initialized opponents:

```
weights = np.load(weights_path)
champ_brain = NeuralNetwork(weights=weights)
champ = Agent(0, 0, brain=champ_brain.copy())
opponents = [Agent(0, 0) for _ in range(config.POPULATION_SIZE - 1)]
```

The champion respawns each round against new random agents, and the console reports whether it survived and its final fitness score. This makes it straightforward to compare a gen 5 champion against a gen 50 champion by running two replay windows side by side.

```
python main.py --replay champions/gen_050.npy
```

```
PS C:\Users\Rohan\Documents\evoarena> python .\main.py --replay .\champions\gen_100.npy
```

Fig 2: Replay command loading a saved champion's weights

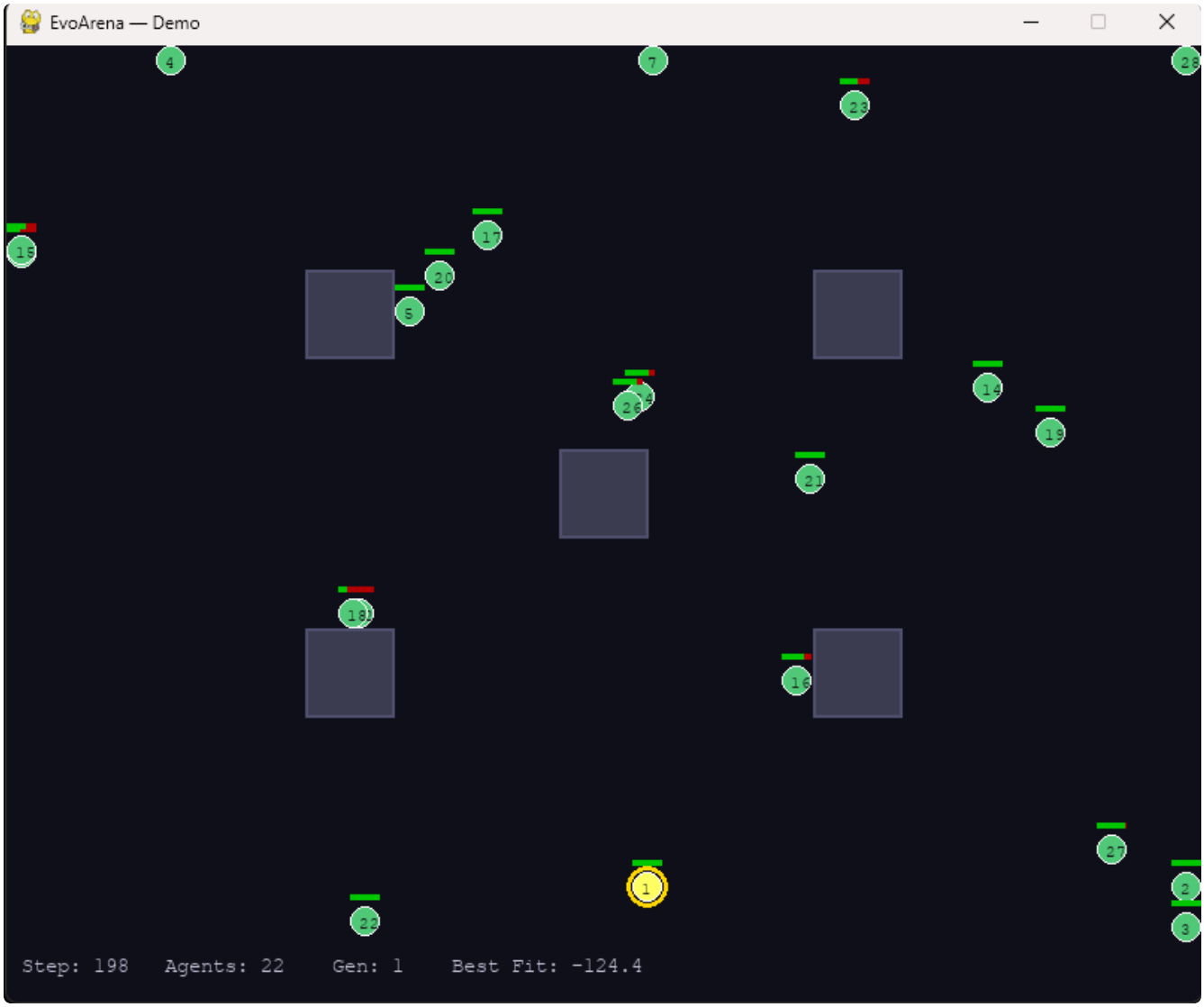

Fig 3: Champion (gold ring) competing against fresh random opponents in replay mode

Pinned champion visual: An issue with the initial replay implementation was that the gold ring, previously always drawn around the highest-fitness agent, would drift to random opponents as they accumulated steps and damage, making it impossible to track the loaded champion visually.

The fix was to add a `pinned_champion_id` attribute to the `Visualizer`. When set, the alpha selection skips the fitness comparison entirely and highlights that specific agent:

```
if self.pinned_champion_id is not None:
    for agent in self.arena.agents:
        if agent.id == self.pinned_champion_id:
            best_agent = agent
            break
```

The replay function sets this immediately after spawning the champion, so the gold ring stays locked to it for the entire run regardless of fitness standings.

3.3 Parameter Sensitivity Analysis

With headless mode available, it became practical to run systematic experiments across parameter configurations. A dedicated script, `experiments/sweep.py`, was written to sweep one parameter at a time while holding the others at their defaults.

The three parameters swept, and the values tested for each:

Parameter	Values Tested	Default
Mutation rate (σ)	0.05, 0.1, 0.2	0.1
Population size	15, 30, 50	30
Elite percentage	10%, 20%, 30%	20%

Each configuration runs for 50 generations headlessly. Best and average fitness are recorded per generation, saved to a JSON file, and rendered as a side-by-side comparison plot. The one-at-a-time methodology keeps each result interpretable, when population size is being swept, mutation rate and elite percentage are fixed at their defaults, so any difference in the curves is attributable solely to population size.

The core run function patches `config` values before each run to avoid modifying the existing codebase:

```
def run(population_size, mutation_rate, elite_pct, generations=50):
    config.POPULATION_SIZE = population_size
    config.MUTATION_RATE   = mutation_rate
    # ... simulation loop
```

---

4. Parameter Sweep Results

4.1 Mutation Rate

Fig 4: Best and average fitness across 50 generations for mutation rates $\sigma \in \{0.05, 0.1, 0.2\}$

All three mutation rates reach a similar best fitness ceiling by generation 50 (~1000–1350), but they differ in how they get there and in average population quality.

Low mutation (σ = 0.05): The best fitness stabilizes quickly but the average fitness lags behind the other two, finishing at 241.8. With small perturbations, children stay close to their parents — the elite lineage is preserved well but the rest of the population is slow to improve, since mutations are too conservative to explore new weight configurations.

Default mutation (σ = 0.1): Average fitness reaches 377.4 by generation 50, outperforming the low mutation run. The additional noise is enough to help non-elite agents escape poor weight configurations without destabilizing the elite lineage.

High mutation (σ = 0.2): Achieves the highest average fitness (395.7) and the highest peak best fitness (1408.1). The larger perturbation introduces more diversity each generation, which appears beneficial at this stage, the

population has not fully converged, so there is still fitness landscape left to explore. The slight edge over σ = 0.1 suggests the default could be nudged upward.

This aligns with the convergence finding from Report 2: crossover's benefit diminishes as diversity collapses, and the same principle applies to mutation magnitude. When diversity is still present, more exploration pays off.

4.2 Population Size

Fig 5: Best and average fitness across 50 generations for population sizes N ∈ {15, 30, 50}

Population size has the most pronounced effect of the three parameters swept.

Small population (N = 15): Best fitness peaks at 1443.6 but ends at 1096.0. Average fitness reaches 440.6, not bad, but with only 3 elites per generation (20% of 15), the diversity available for crossover is limited and the population converges rapidly. Small populations also amplify variance, a single unlucky generation can set the lineage back significantly.

Default population (N = 30): Both best (1672.5) and average (751.8) fitness are clearly the highest of the three configurations. Six elites per generation provides enough diversity for crossover to remain effective through all 50 generations. This confirms the decision made at the midterm to increase population from 10 to 30.

Large population (N = 50): Counterintuitively, the largest population performs worse than N = 30, finishing with best fitness 1197.6 and average 306.7. With 50 agents competing, combat becomes more chaotic — agents encounter more opponents per step, which increases the role of positioning luck over evolved strategy. The elite pool of 10 agents is larger in absolute terms but is drawn from a noisier competitive environment, diluting selection quality.

The result suggests N = 30 is a genuine optimum for this arena size and generation length, not just a default that happens to work.

4.3 Elite Percentage

Fig 6: Best and average fitness across 50 generations for elite percentages ∈ {10%, 20%, 30%}

The elite percentage results show an interesting variance-stability tradeoff.

10% elitism: The most stable configuration, best fitness ends at 1154.0 and average at 373.0. High selection pressure means only the best 3 agents out of 30 contribute to the next generation, which keeps the elite lineage focused but can miss combining complementary strategies from lower-ranked agents.

20% elitism (default): Reaches the highest single-generation peak (1619.7) but ends lower at 875.8, with average fitness going negative by the final generations (-598.8). The larger elite pool enables more diverse crossover early on, but as the pool converges to similar strategies, crossover produces children that are indistinguishable from single-parent mutation, and the population quality degrades late in the run.

30% elitism: Middle ground in outcome, peak of 1348.7, final best of 1015.4, average of -56.0. A larger elite pool preserves more of the population's variation but dilutes selection pressure, slowing early improvement.

The key observation is that 20% elitism reaches the highest peaks but is least stable late in the run. This supports the Report 2 finding that crossover's advantage is largest when diversity is high. A potential improvement would be adaptive elitism,starting with a larger pool (30%) for diverse early exploration and narrowing it (10%) as the population converges.

---

5. Challenges and Solutions

5.1 Pinned Champion Visual

The initial replay implementation relied on the same fitness-based alpha selection used in the normal simulation. During replay, the loaded champion starts each round with zero accumulated fitness (no prior kills or damage), while random opponents quickly accumulate steps_alive points. Within the first 100 steps of any replay round, the gold ring would drift to whichever random agent had survived the longest, making the champion impossible to visually track.

The solution was to add a `pinned_champion_id` field to `Visualizer`. When set, the alpha selection bypasses the fitness comparison and locks directly to the agent with that ID. This required no changes to the simulation logic, only the rendering layer needed to be aware of the pin.

5.2 Config Patching in Sweep

The simulation code reads parameters directly from `config` rather than accepting them as function arguments. To run different configurations in the sweep without refactoring the existing codebase, the sweep script patches `config` values directly before each run:

```
config.POPULATION_SIZE = population_size
config.MUTATION_RATE   = mutation_rate
```

This works cleanly because each run is sequential and Python module state persists within a process. The tradeoff is that it is not thread-safe, parallel sweeps on different threads would corrupt each other's config state. For the current sequential implementation this is not an issue, but parallelizing the

sweep in the future would require either passing parameters explicitly or using multiprocessing with separate interpreter instances.

---

6. Timeline Comparison

The original plan for Week 13 outlined three deliverables:

- Parameter sensitivity analysis, systematically vary mutation rate, population size, and fitness weights
- Run extended simulations (100+ generations) under varied parameters
- Save and replay champion agents from different generations

All three have been addressed. The parameter sweep covers mutation rate, population size, and elite percentage across 50 generations per configuration. Champion save and replay are fully implemented. The 50-generation run length was chosen over 100+ to keep sweep time practical while still capturing the convergence behavior of interest, the key dynamics (early improvement, plateau, convergence) are all visible within 50 generations.

---

7. Next Steps

Week 14:

- Final written report covering system design, algorithms, experiments, and results across all three milestones

- Polish GitHub repository: clean README, inline documentation, reproducibility instructions
- Final video presentation demonstrating multi-generation behavioral evolution, champion replay comparison, and parameter sweep findings

---

8. Tools Usage Acknowledgment

AI tools were used to assist with code suggestions, debugging, and documentation drafting throughout this project. All AI-assisted outputs were reviewed, modified, and integrated with full understanding of the implemented algorithms. This is documented in the GitHub repository README as required.

This report is written in Markdown and LaTeX in an online editor called blankt, which I made.